

Product Requirements Document (PRD)

MVP SaaS Personal Budgeting

Status: **Draft V1.0**

Peran: Senior Technical PM & Cloud Architect

Tanggal: Juni 2026

Teknologi: Vite + React.js | Supabase Cloud (PostgreSQL) | Vercel CI/CD

1. Problem Statement & Value Proposition

Problem Statement

Mengelola keuangan pribadi seringkali menjadi tugas yang menakutkan bagi banyak individu. Pengguna umumnya terjebak pada dua masalah utama:

- Kompleksitas & Inefisiensi:** Penggunaan lembar sebar (spreadsheet) manual memakan waktu lama, kaku, dan rawan kesalahan input. Di sisi lain, aplikasi finansial yang ada di pasar saat ini seringkali terlalu rumit, penuh dengan fitur yang tidak relevan, serta membebani pengguna dengan antarmuka yang membingungkan.
- Kekhawatiran Privasi & Keamanan Data:** Data keuangan bersifat sangat sensitif. Pengguna merasa ragu untuk mengadopsi platform pihak ketiga karena adanya ketakutan bahwa data finansial mereka dapat diakses oleh pihak luar, bocor ke tenant lain, atau disalahgunakan akibat buruknya sistem keamanan siber.

Value Proposition

Aplikasi SaaS Personal Budgeting ini menawarkan platform pencatatan keuangan yang **ringan, cepat, dan memiliki tingkat keamanan kelas enterprise** untuk pasar retail.

Melalui arsitektur **Multi-Tenant SaaS**, setiap pengguna (tenant) mendapatkan ruang kerja (workspace) digital yang terisolasi secara mutlak. Dengan memanfaatkan fitur **Row Level Security (RLS)** bawaan dari PostgreSQL di Supabase Cloud, sistem menjamin keamanan data langsung pada lapisan database terendah, memastikan tidak akan terjadi kebocoran data antar-tenant. Dikombinasikan dengan ekosistem Vercel dan Vite + React.js, platform ini menyajikan pengalaman pengguna yang sangat responsif, instan, layaknya aplikasi desktop lokal tanpa mengorbankan integritas data sedikit pun.

2. Minimum Viable Product (MVP) Features

Untuk memastikan kecepatan penetrasi pasar (time-to-market) dan menjaga fokus pengembangan, cakupan fitur MVP ini secara ketat dibatasi pada tiga fungsi inti berikut:

A. Autentikasi & Registrasi Tenant

Modul ini bertanggung jawab untuk mengatur pintu masuk pengguna dan memastikan pemisahan identitas (tenant isolation) sejak awal.

- **Sign Up / Sign In:** Pengguna dapat melakukan pendaftaran dan masuk menggunakan kombinasi Email dan Password yang dikelola secara aman oleh Supabase Auth.
- **Tenant Workspace Creation:** Begitu proses registrasi berhasil, Supabase Auth secara otomatis menghasilkan UUID (Universally Unique Identifier) yang unik. UUID ini akan bertindak sebagai `tenant_id` permanen untuk mengisolasi seluruh objek data yang dibuat oleh pengguna tersebut.
- **Session Management:** Autentikasi dipertahankan menggunakan JSON Web Tokens (JWT) yang aman, disimpan di sisi client (Vite + React.js), dan diperbarui otomatis untuk mencegah pembajakan sesi.

B. Manajemen Transaksi

Fitur inti untuk melakukan operasi komputasi finansial harian pengguna dengan pendekatan CRUD (Create, Read, Update, Delete) yang cepat.

- **Pencatatan Transaksi Cepat:** Formulir input minimalis untuk mencatat Pemasukan (Income) dan Pengeluaran (Expense) secara instan.
- **Kategorisasi Dinamis:** Setiap transaksi wajib dikaitkan dengan satu kategori (misalnya: Makanan, Transportasi, Gaji, Investasi). Pada fase MVP, sistem menyediakan sekumpulan kategori bawaan (default) yang otomatis terikat dengan `tenant_id` pengguna baru.
- **Daftar Riwayat (Ledger):** Tampilan tabel riwayat transaksi yang mendukung pengurutan (sorting) berdasarkan tanggal dan penyaringan (filtering) sederhana berdasarkan tipe aliran kas.

C. Ringkasan Dashboard Finansial

Menyajikan wawasan finansial (financial insights) secara instan dan visual begitu pengguna berhasil masuk ke sistem.

- **Metrik Arus Kas (Cash Flow Metrics):** Kartu ringkasan (summary cards) yang menampilkan Total Pemasukan, Total Pengeluaran, dan Saldo Bersih (Net Balance) pada bulan berjalan.
- **Visualisasi Grafik:** Grafik lingkaran (Pie Chart) atau batang (Bar Chart) menggunakan pustaka komponen berbasis React untuk mengelompokkan pengeluaran berdasarkan kategori secara proporsional.
- **Indikator Batas Anggaran (Budget Limit Indicator):** Batang kemajuan (progress bar) visual yang membandingkan akumulasi pengeluaran bulan ini dengan batas anggaran global yang ditetapkan oleh pengguna, memberikan peringatan visual jika mendekati batas.

3. Multi-Tenant User Flow Diagram Deskriptif

Berikut adalah visualisasi alur logis langkah-demi-langkah dari perspektif arsitektur keamanan data terisolasi:

Langkah 1: Akses & Pendaftaran (Frontend ke Auth Layer)

Pengguna mengakses aplikasi web melalui URL yang di-host di Vercel. Pengguna mengisi form pendaftaran. Frontend React mengirimkan kredensial ke Supabase Auth API via HTTPS. Supabase memvalidasi, membuat user record baru, dan menghasilkan nilai `auth.uid()` yang unik. Sesi JWT dikembalikan ke browser pengguna.

Langkah 2: Inisialisasi Profil (Database Layer)

Sebuah trigger otomatis di dalam PostgreSQL Supabase menangkap pembuatan user baru tersebut dan langsung menyisipkan baris data baru ke tabel `public.profiles` dengan menyamakan ID profil dengan `auth.uid()`. Ini mengunci identitas tenant di level database.

Langkah 3: Operasi Data Keuangan (Strict Isolation Check)

Saat pengguna mencatat pengeluaran baru, frontend mengirimkan muatan data (jumlah, kategori, tanggal) bersama dengan JWT di dalam header HTTP Authorization. Gateway Supabase mengurai JWT tersebut untuk mengekstrak identitas pengguna. RLS (Row Level Security) mencegah operasi ini dan memastikan kolom `tenant_id` pada baris yang dimasukkan wajib sama dengan `auth.uid()`. Manipulasi ID dari sisi client ditolak mentah-mentah.

Langkah 4: Pemanggilan & Agregasi Dashboard (Sistem Kuat Tanpa Bocor)

Saat memuat halaman dashboard, query `SELECT * FROM transactions` dijalankan oleh frontend. Lapisan RLS PostgreSQL secara otomatis mengintersept query tersebut dan menyisipkan klausa implisit: `WHERE tenant_id = auth.uid()`. Akibatnya, database hanya mengolah dan mengembalikan data milik tenant aktif tersebut. Kebocoran data tidak mungkin terjadi meskipun ada bug pada kode frontend.

Langkah 5: Terminasi Sesi (Sign Out)

Pengguna menekan tombol "Logout". Sesi di dalam state React dihancurkan, token JWT dihapus dari local storage browser, dan koneksi ke Supabase client ditutup sepenuhnya.

4. Database Schema PostgreSQL for Supabase

Rancangan skema relasional ini dioptimalkan untuk performa tinggi pada Supabase, memanfaatkan relasi foreign key yang ketat ke tabel autentikasi internal.

Tabel: `profiles`

Menyimpan data entitas utama dari tenant yang terdaftar di aplikasi.

Nama Kolom	Tipe Data	Constraint / Keterangan
<code>id</code>	<code>uuid</code>	PRIMARY KEY, REFERENCES <code>auth.users(id)</code> ON DELETE CASCADE. Bertindak sebagai ID Tenant global.
<code>email</code>	<code>text</code>	NOT NULL, Menyimpan alamat email pengguna untuk keperluan sinkronisasi UI.
<code>full_name</code>	<code>text</code>	NULL, Nama lengkap atau nama tampilan pengguna di aplikasi.
<code>created_at</code>	<code>timestamp</code>	NOT NULL, DEFAULT <code>now()</code> . Waktu pendaftaran akun.

Tabel: `categories`

Menyimpan kategori transaksi keuangan yang bersifat spesifik per tenant.

Nama Kolom	Tipe Data	Constraint / Keterangan
<code>id</code>	<code>uuid</code>	PRIMARY KEY, DEFAULT <code>gen_random_uuid()</code> .
<code>tenant_id</code>	<code>uuid</code>	NOT NULL, REFERENCES <code>profiles(id)</code> ON DELETE CASCADE. Penanda kepemilikan tenant.
<code>name</code>	<code>text</code>	NOT NULL. Nama kategori (contoh: "Makanan", "Gaji").
<code>type</code>	<code>text</code>	NOT NULL. Batasan nilai: 'INCOME' atau 'EXPENSE'.
<code>created_at</code>	<code>timestamp</code>	NOT NULL, DEFAULT <code>now()</code> .

Tabel: `transactions`

Tabel transaksional utama untuk melacak seluruh arus masuk dan keluar dana keuangan.

Nama Kolom	Tipe Data	Constraint / Keterangan
<code>id</code>	<code>uuid</code>	PRIMARY KEY, DEFAULT <code>gen_random_uuid()</code> .
<code>tenant_id</code>	<code>uuid</code>	NOT NULL, REFERENCES <code>profiles(id)</code> ON DELETE CASCADE. Kolom isolasi tenant utama.
<code>category_id</code>	<code>uuid</code>	NOT NULL, REFERENCES <code>categories(id)</code> ON DELETE RESTRICT. Relasi ke kategori terkait.
<code>type</code>	<code>text</code>	NOT NULL. Denormalisasi tipe untuk performa: 'INCOME' atau 'EXPENSE'.
<code>amount</code>	<code>numeric</code>	NOT NULL. Menggunakan tipe <code>numeric</code> tanpa floating-point untuk presisi perhitungan finansial.
<code>description</code>	<code>text</code>	NULL. Catatan tambahan dari pengguna mengenai transaksi tersebut.
<code>trx_date</code>	<code>timestamp</code>	NOT NULL. Tanggal aktual terjadinya transaksi keuangan.
<code>created_at</code>	<code>timestamp</code>	NOT NULL, DEFAULT <code>now()</code> . Tanggal sistem mencatat entri.

5. Security & RLS Blueprint Sederhana

Lapisan pertahanan utama dari arsitektur multi-tenancy ini bertumpu pada Row Level Security (RLS) di dalam mesin PostgreSQL Supabase. Kode cetak biru di bawah ini wajib dieksekusi pada database produksi untuk menjamin isolasi data mutlak:

Mengaktifkan RLS

```
-- Aktifkan kontrol RLS secara eksplisit pada tabel transaksional
ALTER TABLE public.transactions ENABLE ROW LEVEL SECURITY;
```

Definisi Kebijakan Keamanan (Security Policies)

Fungsi `auth.uid()` adalah fungsi internal Supabase yang secara aman mengekstrak UUID pengguna langsung dari token JWT yang telah diverifikasi melalui algoritma kriptografi di sisi server. Hal ini menjamin parameter tersebut tidak dapat dipalsukan.

1. Kebijakan Membaca Data (SELECT):

Membatasi hak akses baca sehingga pengguna hanya dapat melihat baris data yang memiliki nilai `tenant_id` sama dengan ID sesi aktif mereka.

```
CREATE POLICY "Tenant can view own transactions"
ON public.transactions
FOR SELECT
USING (auth.uid() = tenant_id);
```

2. Kebijakan Menambahkan Data (INSERT):

Memastikan pengguna hanya diperbolehkan memasukkan baris transaksi baru jika mereka menyertakan ID mereka sendiri ke dalam kolom `tenant_id`.

```
CREATE POLICY "Tenant can insert own transactions"
ON public.transactions
FOR INSERT
WITH CHECK (auth.uid() = tenant_id);
```

3. Kebijakan Mengubah Data (UPDATE):

Menjamin operasi pembaruan data hanya diizinkan pada baris milik sendiri, dan melarang pemindahan kepemilikan data ke tenant lain.

```
CREATE POLICY "Tenant can update own transactions"
ON public.transactions
FOR UPDATE
USING (auth.uid() = tenant_id)
WITH CHECK (auth.uid() = tenant_id);
```

4. Kebijakan Menghapus Data (DELETE):

Memastikan operasi penghapusan data secara destruktif dibatasi ketat hanya untuk pemilik baris data yang sah.

```
CREATE POLICY "Tenant can delete own transactions"
ON public.transactions
FOR DELETE
USING (auth.uid() = tenant_id);
```

Catatan Tambahan Cloud Architect: Pola arsitektur empat kebijakan keamanan di atas juga wajib direplikasi secara identik pada tabel `public.profiles` dan `public.categories` guna mengunci seluruh ekosistem aplikasi dari risiko kebocoran data silang.